

Could Global Software Development Benefit from Agile Methods?

Maria Paasivaara and Casper Lassenius
Software Business and Engineering Institute
Helsinki University of Technology
P.O.Box 9210 FIN-02015 TKK, Finland
Maria.Paasivaara@tkk.fi, Casper.Lassenius@tkk.fi

Abstract

At first glance, agile methods and global software development might seem incompatible. Agile methods stress continuous face-to-face communication, whereas communication has been reported as the biggest problem of global software development. One challenge to solve is how to apply agile practices in settings where continuous face-to-face interaction is missing. However, agile methods have been successfully used in distributed projects, indicating that they could benefit global software development. This paper discusses potential benefits and challenges of adopting agile methods in global software development. The literature on real industrial case studies reporting on experiences of using agile methods in distributed projects is still scarce. Therefore we suggest further research on the topic. We present our plans for research in companies using agile methods in their distributed projects. We also intend to test the use of agile principles in globally distributed student projects developing software for industrial clients.

1. Introduction

Global software development, including outsourcing, subcontracting and partnerships, is becoming increasingly common. However, software projects developing genuinely novel products are often faced with uncertainty regarding, e.g., both requirements and implementation technologies, and subcontractors or partners need to be involved long before these uncertainties can be resolved. In such projects, the parties cannot receive clear requirement specifications at the beginning. Instead, close cooperation and communication between the parties is required during the whole project, as the project both builds a product and tries to understand what to build at the same time. Another trend, the usage of agile development methodologies in software development, emphasizes communication and close collaboration. Such methodologies are thus particularly suitable for development projects facing high degrees of uncertainty. Many of the agile practices are built on face-to-face communication and collocation is assumed. Thus, even though these two trends seem incompatible, there are

some elements that connect them, e.g., the need for frequent communication [11, 17]. Even though most agile methods have been designed for collocated usage and their practices typically require face-to-face interaction, agile methods, such as Scrum [15] and XP [2], have been successfully used in distributed projects (e.g., [4, 5, 11, 17]). These experiences indicate that agile methods could help solve some problems of distributed development by providing practices that can be applied in distributed settings. However, there are still many challenges to be solved to be able to apply agile methods to globally distributed projects, where collocation and continuous face-to-face interaction between developers is missing.

This paper discusses the potential benefits of adopting agile methods in distributed software development projects and the unsolved challenges that agile methods bring along. At the moment, the literature on the usage of agile methods in distributed projects is scarce. There are few reported experiences on applying agile methods to distributed industrial projects, even though we know that several companies have carried out that kind of experiments. In this paper we first discuss the benefits and challenges of applying agile methods in distributed projects based on current literature. Then, we suggest further research ideas on the topic. Finally, we present our own research plans to study both industrial projects using agile methods and testing the usage of agile principles in globally distributed student projects developing software for industrial clients.

2. Combining distributed development and agile methods

Global software development literature brings up many challenges related to distributed development, e.g., interdependencies among work items that are distributed, difficulties of coordination, difficulties of dividing the work into modules that could be assigned to different locations, conflicting implicit assumptions that are not noticed as fast as in collocated work, and communication challenges [10]. Literature suggests dividing the work into separate modules that then can be distributed to different sites to be developed [7]. These modules should be independent in order to minimize communication between sites [7]. The authors

emphasize that it is possible to split only well-understood products for which architecture and plans are likely to be stable. However, in the current development environment with a lot of uncertainties, dividing the software into modules and specifying the modules in detail upfront is often impossible. Moreover, this way of development, specifying and dividing work first and then in the end integrating all in “a big bang”, is challenging, since integration can cause huge problems, that may not have been expected. As a solution Battin et al. [1] suggest an incremental integration plan, which would be based on clusters and shared incremental milestones to avoid “big bang” integration. This strategy was used successfully at Motorola. Ebert and De Neve [3] provide similar experiences about the usage of incremental development in Alcatel, where they developed each increment within one dedicated team and based their progress tracking on successfully integrated and tested customer requirements. The authors report that a stable build proved to be one of the key success factors and that globally applied continuous builds improved project cycle time. Neither the Alcatel nor the Motorola case report the integration interval, but it seems these cases did not use very frequent regular build cycles, such as daily or weekly builds. However, even very frequent builds are possible in distributed development. Karlsson et al. [8] report their experiences of using daily builds and feature-based development successfully in distributed projects.

Incremental integration and frequent deliveries is a core practice in agile methodologies [9] but its use in distributed development has not received much attention yet. Fowler [5] and Simons [17] reported their experiences on using agile methods in offshore software development projects. According to Simons [17] an iterative model seems to work well in distributed projects and can eliminate some of the problems that distribution brings. Continuous integration and build verification tests solve integration problems in small steps and help avoid large integration problems at the end of the project. Moreover, iterative development with frequent deliveries provides increased visibility into project status, which makes it easier for project managers and customers to follow project progress [17]. Fowler [5] discusses suitable iteration lengths for offshore projects and concludes that two-week iterations are minimum, because of the communication overheads of distributed development, and two to three month iterations start already to be too long. Moreover, our earlier research [12, 13] studying five inter-organizationally distributed software development projects using iterative and incremental development showed that iterative development and frequent deliveries are beneficial to distributed projects. Thus, it seems that at least the agile principles of frequent deliveries and continuous integration seem to suit global software development. Both Fowler [5] and Simons [17] emphasize that main benefits of using agile development in their projects have been the high responsiveness to change and fast delivery of

business value, and that these benefits have outweighed the challenges of distributed development.

2.1 Distributed extreme programming

A few studies have presented experiences on using extreme programming (XP) in distributed software development projects. Both Fowler [5] and Simons [17] give quite detailed descriptions of their experiences of using XP in large projects distributed between the US and India. Both onshore and offshore teams used agile methods and the projects applied the agile communication principles in these highly distributed settings. The projects successfully used practices such as continuous integration, short iterations, regular short status meetings, multiple communication modes, proxy customers in offshore teams and resource rotation. Nisar and Hameed [11] and Xiaohu et al. [18] describe their experiences on using XP in offshore teams collaborating with onshore customers. Both papers discuss projects where the development work is done in offshore teams only but the onshore customer is tightly involved in project communication. Nisar and Hameed [11] report briefly the main agile principles they followed and emphasize that client satisfaction is the main reason to use XP, which has proven to be successful in their offshore projects. For the project presented by Xiaohu et al. [18] the main reason for implementing XP practices was to reduce the communication delay and improve communication quality between the customers and the offshore development team. Both Nisar and Hameed [11] and Xiaohu et al. [18] conclude that the reported projects have been very successful and the XP principles they have followed have proven to work. Karlsson et al. [8] and Farmer [4] report their experiences on using a few XP practices (e.g., continuous builds and unit tests [4] and planning game, small releases, continuous integration and automatic testing [8]) in large distributed projects. Both papers found the applied agile practices useful but hard to implement. All these reported experiences on the usage of XP in distributed projects were successful according to the respective authors. This leads us to a conclusion that XP principles could be useful in distributed projects, even though a lot of work is needed in designing how to implement the XP principles in distributed settings.

2.2 Distributed Scrum

In his book Schwaber [16] presents how the Scrum method can be scaled to large projects involving multiple Scrum teams. He presents some scaling mechanisms, such as daily scrum of scrums for coordinating the work of Scrum teams. According to Schwaber [16] these scaling mechanisms enable the usage of Scrum also in geographically distributed projects. He suggests that in these kinds of projects high-bandwidth technology for source code sharing, synchronized builds and alternative communication

methods such as instant messaging need to be employed. Besides the experiences described by Schwaber [16], we did not find experience reports on using Scrum in distributed projects from the literature, but we have encountered its usage in a few distributed industrial projects. In one of the projects a Finnish customer company was developing software using both several onsite subcontractors and a subcontractor that had both onsite developers working in the customer's premises and offshore developers. Earlier, the company had used a waterfall type process with its main subcontractor. This development model had many problems, which led the company to experiment with the usage of Scrum. The customer company had applied Scrum practices to suit its needs, especially communication practices and tools had received special attention. Local developers and offshore developers worked normally in separate teams consisting approximately of six developers. When new knowledge, e.g., new practices needed to be transferred to offshore teams, some persons from the offshore teams were invited to work locally as members of onshore teams during one sprint. Otherwise mainly electronic communication was used, e.g. mailing lists, Wiki, IRC, Messenger, Skype and tele- and videoconferencing. Project documents and reports were stored in the Wiki and daily communication between the teams took place through IRC, Messenger and Skype. The idea was to keep communication as visual as possible, thus, e.g. daily scrums of scrums were arranged using videoconference if possible. The project used two week sprints which ended with sprint demos in which all teams presented their results. Offshore developers participated in the sprint demos through videoconferencing and desktop sharing. Sprint demos were open to all stakeholders and they were seen as one true measure of completion. The project used test-driven development and continuous integration. Fast communication and reaction to any integration problems were seen as important drivers of success. Since the project was developing a product to be sold to several customers, a real end customer could not be involved. Instead, the customer company's system architect took the role of proxy customer to the offshore teams. He was regarded as capable of both talking about technical issues and passing on business requirements. In summary, this still on-going distributed project was successfully using Scrum principles in a project that was both geographically and organizationally distributed. The project had invested a lot of effort in developing communication practices to build and maintain frequent communication between onshore and offshore teams. Even though the project had experienced some communication problems with the offshore teams, such as misunderstandings of the requirements or developers not asking for help fast enough when having problems, the project was so far seen as very successful. Thus, based on limited experiences it seems that it is possible to apply Scrum principles to distributed projects when enough attention is paid on planning how to arrange frequent communi-

cation efficiently. Of course, this is just one case project, thus more reported industrial experiences are needed. It would be particularly interesting to gather more detailed information about communication arrangements in other distributed projects using Scrum.

2.3 Challenges of distributed agile methods

There are several challenges for the application of agile methods to global software development, communication probably being the most obvious one. In the global software development literature, communication is often seen as the most challenging problem of distribution (e.g. [10]) whereas the agile methods basically rely on communication, preferably face-to-face communication, instead of documentation. The application of agile principles to global software development poses several questions regarding communication: How could daily communication be arranged effectively? What kind of communication practices and media are suitable for supporting different agile practices? How could informal communication, that is important to agile methods, be encouraged? How could the risk for misunderstandings, e.g. regarding requirements, be minimized? How could trust be build and retained between teams to ensure open communication?

Agile methods use short iterations, frequent builds, and continuous integration. These practices pose challenges to configuration management and version management. Especially when the distributed teams come from different companies, e.g., the customer company is using subcontractors, using a common tool or set of tools might be challenging to arrange. Moreover, long distances and bad infrastructures might cause extra challenges. This poses a question: How could these practices be made possible and functioning in globally distributed projects both regarding the technical aspects and trust issues between the companies?

Agile methods emphasize change; the business environment and customer requirements are not stable, but changing. Thus, agile methods see frequent communication with customers as important. However, in distributed development, the customers may be located far away making frequent interaction difficult to arrange. For example, an XP practice, on-site customer, might be challenging to arrange when distances are large and if an on-site customer can be arranged he or she might gradually lose connection to his or her own environment.

Cultural challenges are common to global software development. Introducing an agile method can change the command and control model in a company; developers need more autonomy and decision-making power than what they are used to, to be able to implement the agile practices [5].

2.4 Possible benefits of distributed agile methods

Besides being the biggest challenge, communication can also be seen as a major benefit of agile methods, since they explicitly emphasize communication and provide many useful communication practices. Even though the practices are originally designed to be used in collocated development, many practices seem to be transferable to distributed development, when suitable tools are found, as some successful examples in the literature show (e.g. [5, 11]). Thus, the strong emphasis on frequent communication can be regarded as strength.

Also short iterations, frequent builds, and continuous integration emphasized by agile methods can be seen as benefits in addition to being challenges. The benefits of short iterations are numerous [5, 13]: They bring transparency of work progress to all partners. Both the developers, project managers and customers can frequently get a good picture of how the project is progressing. While the customer can monitor real progress, the distributed developers can get instant feedback on their work, which is motivating. Moreover, seeing high quality work early and frequently builds trust and respect between distributed partners making further collaboration easier.

From the customer's point of view this way of developing brings additional flexibility when the customer can do changes during the development phase without time consuming negotiations with subcontractors. It also enables the customer company to bring subcontractors into the project already in the early phases of development, when the customer is not yet able to specify the requirements in detail but can rely on early correction of direction when needed. Frequent integration and testing also ensures that all parties have understood the requirements correctly. This is a typical uncertainty in distributed development, especially when participants have not worked together before and come from different cultures. Continuous integration and testing gives fast feedback and any misinterpretations become visible early and possible misunderstandings have less damaging consequences. Moreover, learning from mistakes is fast and happens early, thus preventing problems to accumulate or grow.

Even the problems with cultural differences can be helped by using agile methods [11]: frequent and open communication between the participants and frequent releases to customers build trust and project participants learn quickly more about each others cultures. Thus, the probability of serious cultural problems decreases quickly.

3. Suggestions for future research

The combination of agile methods and distributed development poses several challenges, but also possible benefits. As Section 2 showed, the literature on using agile methods in global software development is still scarce. Thus, more research on this interesting topic would benefit

especially the companies considering using this combination in their software development projects.

3.1 Themes for future research

Below we list a few major questions that arise based upon the challenges and benefits of distributed agile methods:

- How should the agile practices be applied to distributed projects?
- What are the most effective agile practices for distributed use?
- To what kinds of distributed projects are agile methods best suitable?
- What are the biggest challenges in applying agile practices to distributed projects?
- How could these be avoided or alleviated?

3.2 Our plans for future research

We plan to study the usage of agile methods in industrial projects. We are especially interested in how agile methods can be successfully tailored and applied to distributed projects. Moreover, we plan to study what the benefits of using agile methods in distributed projects are and to what kinds of distributed projects they are especially suitable. Finally, we will study what kind of problems or challenges the agile methods bring to distributed projects and how these challenges could possibly be solved. The main research method will be qualitative case study [19]. By using purposeful selection [14] a group of 2-4 case companies using agile methods in their distributed projects will be selected for the study. In particular, we will try to include distributed projects using Scrum, since we did not find any experiences of using that approach in distributed projects in the literature, whereas the literature has reported already a few experiences of using extreme programming in distributed projects (e.g. [5, 11, 17, 18]). The main data collection method regarding this group of companies will be qualitative semi-structured interviews [14].

Besides collecting industrial experiences, we plan to arrange a course on distributed software development. The course has two goals: first, to teach distributed software development to our students, and second to give us a possibility to study how agile practices can be applied to distributed projects. The course is planned to be arranged between three different universities situated in Finland, Brazil and the US. During this course the students are first taught the agile methods they are supposed to use during the course. The methods, practices and tools to be used are selected based on the first part of this research, namely on the interviews in the case companies. At the moment we expect that we will select the Scrum approach that is modified for use in distributed development. The members of each student group are distributed to three different locations. These projects will each have a real customer offering a limited-

scale project. The usage of practices is observed and the communication behavior of the student groups is analyzed based on data collected by surveys and archiving email communication, which will be analyzed using the TeCFlow tool [6]. We hope to be able to find successful practices and challenges, as well as improvement ideas.

4. Conclusions

At first glance, agile methods and global software development might seem incompatible, since communication is often reported as the biggest problem of distributed development, whereas frequent collaboration and face-to-face communication belong to the basic principles of agile methods. However, there is a possibility that this problem could be turned into a solution: agile methods could provide communication practices that could, with some applying, be used to satisfy the communication needs of distributed development. The literature on real industrial case studies reporting experiences on the usage of agile methods in distributed projects is still scarce; therefore this paper suggests further research on this topic. In particular, experiences on how to apply agile methods to distributed projects and how to accomplish functioning communication would be interesting, but also studies reporting the challenges would be valuable.

Acknowledgements

The authors gratefully acknowledge the financial support of the Academy of Finland (Grant No. SA 107636, 2005-2007).

References

- [1] Battin, R.D., Crocker, R., Kreidler, J., and Subramanian, K. Leveraging resources in global software development. *IEEE Software*, (March/April 2001), 70-77.
- [2] Beck, K. *Extreme Programming Explained: Embrace Change*. Addison Wesley, New York, 2000.
- [3] Ebert, C. and De Neve, P. Surviving Global Software Development. *IEEE Software*, (March/April 2001), 62-69.
- [4] Farmer, M. (2004) DecisionSpace Infrastructure: Agile Development in a Large, Distributed Team. *Proceedings of the Agile Development Conference 2004*.
- [5] Fowler, M. Using Agile Software Process with Offshore Development. (<http://martinfowler.com/articles/agileOffshore.html>) 7.1.2004
- [6] Gloor, P. Laubacher, R. Dynes, S. and Zhao, Y. Visualization of Communication Patterns in Collaborative Innovation Networks: Analysis of some W3C working groups. *Proceedings of ACM CKIM International Conference on Information and Knowledge Management*, (New Orleans, Nov 3-8, 2003).
- [7] Herbsleb, J. and Grinter, R. Architectures, coordination, and distance: Conway's law and beyond. *IEEE Software*, 16, 5, (Sept.-Oct. 1999), 63 -70.
- [8] Karlsson, E., Andersson, L. and Leion, P. Daily Build and Feature Development in Large Distributed Projects. *Proceedings of the International Conference on Software Engineering*, (Limeric, Ireland, 2000), 649-658.
- [9] Larman, C. and Basili, V. Iterative and Incremental Development: A Brief History. *IEEE Computer*, (June 2003), 47-56.
- [10] Mockus, A. and Herbsleb, J. Challenges of Global Software Development. *Proceedings of the Seventh International Software Metrics Symposium*, (METRICS 2001, IEEE), 182-184.
- [11] Nisar, M. and Hameed, T. Agile Methods Handling Offshore Software Development Issues. *Proceedings of INMIC 2004, 8th International Multitopic Conference*, Dec.2004. 417-422.
- [12] Paasivaara, M. *Communication Practices in Inter-Organisational Product Development*. Doctoral Dissertation. Helsinki University of Technology, 2005. Available at: <http://lib.tkk.fi/Diss/2005/isbn9512279355/>
- [13] Paasivaara, M. and Lassenius, C. Synchronizing Global Software Development by Using Incremental Development and Frequent Deliveries. *Proceedings of the ICSE International Workshop on Global Software Development*, Edinburgh, May 2004.
- [14] Patton, M. Q. *Qualitative Research and Evaluation Methods*. Newbury Park, CA: Sage Publications. 1990.
- [15] Schwaber, K. and Beedle, M. *Agile Software Development with Scrum*. Prentice-Hall, Upper Saddle River, N.J., 2002.
- [16] Schwaber, K. *Agile Project Management with Scrum*. Microsoft Press, Redmond, Washington, 2004.
- [17] Simons, M. Internationally Agile. *InformIT* (March 15th, 2002)
- [18] Xiaohu, Y., Bin, X., Zhijun, H. Maddineni, S. Extreme Programming in Global Software Development. *Proceedings of the Canadian Conference on Electrical and Computer Engineering*, 2-5 May 2004. 1845 – 1848. Vol.4
- [19] Yin, R.K. *Case Study Research, Designs and Methods*. Thousand Oaks, California: Sage Publications, 1994.